

Cool and Cruel Bit Recovery Attack on LWE

Sparse Binary Secrets and Partial Lattice Reduction

MAMANCHUK Mykola, SID.202420671
Supervising Professor: KUNIHIRO Noboru

University of Tsukuba
Cryptography and Information Security Laboratory

January 27, 2025

Outline

- 1 Introduction
- 2 Core Idea of the Attack
- 3 Complexities & Results
- 4 Attack Overview
- 5 Practical Considerations
- 6 Lattice Reduction
- 7 Statistical Tests
- 8 Attack Performance
- 9 Conclusions
- 10 Recovery Results
 - Comparison with Existing Attacks
 - 2-power Cyclotomic Ring-LWE
- 11 Discussion and Future Work
- 12 Proposal: Cool, Chill, and Cruel

Motivation & Background

Learning With Errors (LWE) is a core problem in lattice-based cryptography, widely used in post-quantum cryptography (PQC).

- **Sparse binary secrets:** e.g., $\mathbf{s} \in \{0, 1\}^n$ with small Hamming weight h .
- **Homomorphic Encryption (HE):** Often assumes small error and sparse secrets for efficiency.
- Traditional attacks do *not* always account for these “special” parameters.

Key Challenge: Evaluate real-world security of LWE with such specialized parameters.

LWE Definition

Search-LWE Problem:

$$\text{Given } \mathbf{A} \in \mathbb{Z}_q^{m \times n}, \quad \mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{q},$$

where

- $\mathbf{s} \in \mathbb{Z}_q^n$ is the secret vector,
- $\mathbf{e} \in \mathbb{Z}_q^m$ is the error (small-norm distribution),
- m is the number of samples, n is dimension, q is a modulus.

Goal: Recover $\mathbf{s}$ given $\mathbf{A}$ and $\mathbf{b}$.

Sparse Binary Setting:

$$\mathbf{s} \in \{0, 1\}^n, \quad \text{Hamming weight} = h \ll n.$$

“Cool and Cruel” Attack Overview

Motivation:

- Perform partial lattice reduction to separate LWE secret bits into:
 - **Cruel bits**: Unreduced, high-variance portion
 - **Cool bits**: Reduced, low-variance portion
- Brute force the small “cruel” portion and then recover the “cool” part easily.

Three-Stage Attack:

- 1 Parallel *partial* lattice reduction,
- 2 Brute force the “cruel” bits,
- 3 Statistically recover the “cool” bits.

Partial Lattice Reduction: Embedding

Consider the $(m + n) \times (m + n)$ matrix

$$\Lambda = \begin{pmatrix} 0 & qI_n \\ \omega I_m & \mathbf{A} \end{pmatrix}.$$

- ω = penalty parameter controlling the trade-off between *reduction quality* and *error amplification*.
- Apply a lattice reduction algorithm (e.g., BKZ, “flatter”).

After reduction,

$$\mathbf{A} \longrightarrow \mathbf{RA} \equiv \mathbf{A}'.$$

The first n_u columns typically remain high-variance, the last $n_r = n - n_u$ are significantly reduced.

Cruel vs. Cool Bits

Cruel bits:

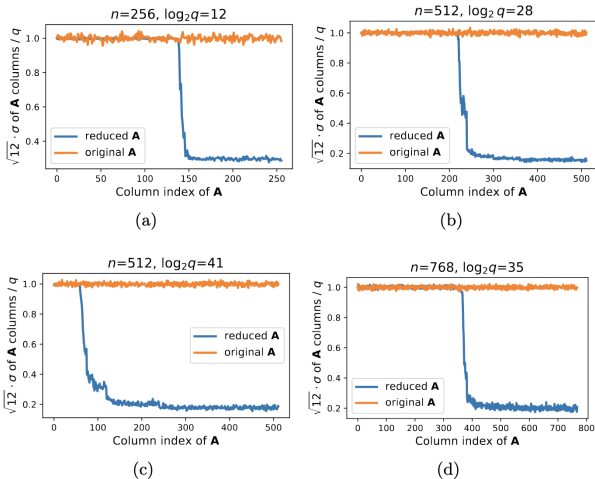
- The n_u unreduced columns (remain $\sim$ uniform in $\mathbb{Z}_q$).
- Hard portion of the secret.

Cool bits:

- The $n_r = n - n_u$ reduced columns (low variance).
- Easy portion of the secret (recoverable with simple stats).

Key Observation: After partial reduction, the dot product $\mathbf{a} \cdot \mathbf{s}$ is dominated by the cruel bits if you guess them incorrectly. The correct guess yields a recognizable residual pattern, making it statistically distinguishable.

Figure: Variance Separation



Conceptual Plot: The original $\mathbf{A}$ (orange) has uniform column variance $\approx q/\sqrt{12}$, while the reduced $\mathbf{A}'$ (blue) shows a significant drop after column index n_u .

Cruel Bit Brute Force

Step 2 of Attack:

- We only guess $\mathbf{s}_{0:n_u}$, the cruel portion.
- Evaluate

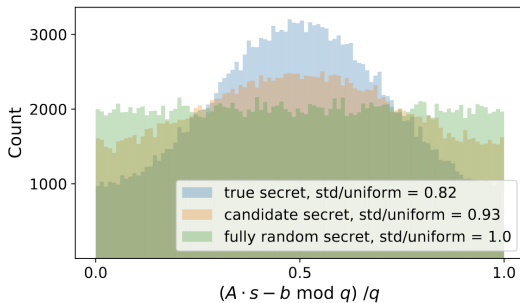
$$\mathbf{RA} \cdot \mathbf{s}^* - \mathbf{Rb} \pmod{q}$$

for each guess $\mathbf{s}^*$ in that subspace.

- The *correct guess* exhibits narrower standard deviation or a distinct residual distribution (as shown in next figure).

Why feasible? n_u is significantly smaller than n ; or the attacker knows h_u (the smaller Hamming weight among the cruel bits).

Figure: Residual Histograms



Blue: true secret residual distribution, **Orange:** partial-correct guess (cruel bits correct), **Green:** random guess. Notice how the orange & green differ in standard deviation or shape.

Cool Bit Recovery

After correctly identifying the n_u cruel bits:

- The last n_r columns are of *low variance*.
- We can fix $\mathbf{s}_{0:n_u}$ and search each cool bit individually.

Greedy Recovery (Algorithm 1):

- For i in $n - n_r : n$, flip bit i and check standard deviation

$$\sigma((\mathbf{A}\mathbf{s}^* - \mathbf{b})).$$

- Keep the bit flip if it *lowers* or does not raise σ significantly.
- Move on to the next bit.

Algorithm 1: Greedy Recovery of Cool Bits

Algorithm Greedy Recovery of Cool Bits

Require: s^* initialized with correct cruel bits (size n_u) and 0 for n_r bits

```

1: for  $i \in \{n - n_r, \dots, n - 1\}$  do
2:    $s^*[i] \leftarrow 0$ 
3:    $\sigma_0 \leftarrow \sigma(\mathbf{A} \cdot s^* - \mathbf{b} \bmod q)$ 
4:    $s^*[i] \leftarrow 1$ 
5:    $\sigma_1 \leftarrow \sigma(\mathbf{A} \cdot s^* - \mathbf{b} \bmod q)$ 
6:   if  $\sigma_0 \leq \sigma_1$  then
7:      $s^*[i] \leftarrow 0$ 
8:   else
9:      $s^*[i] \leftarrow 1$ 

```

Idea: flip each bit individually and choose the value that *minimizes* the residual's standard deviation.

Complexities & Experimental Timings

Three Main Costs:

- ① **Lattice Reduction:** Performed in parallel (e.g., 12+ hours/matrix).
- ② **Brute Force on Cruel Bits:** Dependent on n_u or h_u . (Often feasible if n_u is small)
- ③ **Statistical Recovery of Cool Bits:** Linear or near-linear in n_r .

Reported Timings Example:

n	$\log_2 q$	Hamming Weight	Time (seconds)
256	12	12	3865
512	28	12	2417
	41	60	376
768	35	12	1291
1024	50	17	6395

Why It Works

- 1 **Non-uniform variance** after partial reduction splits the secret bits.
- 2 **Statistical detection** of correct cruel bits via narrower distribution of residual.
- 3 **Low-variance columns** let us do a simple one-bit-at-a-time approach for the cool bits.

Practical Impact:

- Sparse binary (and ternary) secrets are used in real cryptosystems (e.g., HE).
- This attack shows that certain parameter sets are vulnerable if n_u is not too large and the penalty parameter is well-tuned.

Takeaways and Next Steps

Key Points:

- Partial lattice reduction $\rightarrow$ *cruel* vs. *cool* separation.
- Brute force on a small subproblem, then linear-time statistical recovery.
- Attacks extended to RLWE for 2-power cyclotomic rings (rotation trick).

Ongoing & Future Work:

- Further optimization or alternative partial reduction for smaller overhead.
- Combine with other ML-based distinguishers (e.g., SALSA, transformers).
- Evaluate larger n and different error/secret distributions (ternary, Gaussian).

Implementation & Parallel Brute Force

Key Points:

- **Implementation Setup:**

- Written in Python, accelerated with `pytorch` on GPUs.
- Batches of secret candidates evaluated in parallel.

- **Brute Force for Cruel Bits:**

- Enumerate possible secret values for the “cruel” part of $\mathbf{s}$.
- Evaluate residuals $\mathbf{A}\mathbf{s}^* - \mathbf{b} \bmod q$.

- **Ascending Hamming Weight:**

- Start with Hamming weight 0, 1, 2, etc.
- Keep top- k candidates for deeper evaluation.

Parallelizability: Because each candidate check is independent, the brute force step scales linearly with the number of GPUs/cores.

Candidate Evaluation: Variance Test

Residual Distribution:

$$x = \mathbf{A}\mathbf{s}^* - \mathbf{b} \pmod{q}.$$

- If $\mathbf{s}^*$ is *wrong* (cruel bits incorrect) $\implies x \approx \text{uniform mod } q$.
- If $\mathbf{s}^*$ is *correct* (for the cruel portion) $\implies x$ has smaller variance, akin to a “mod-Gaussian.”

Why Variance?

- Non-parametric tests (K-S, Kuiper) can be slow on large N .
- Likelihood tests over modulo Gaussians require summations over $k \cdot q$ shifts.
- **Simple approach:** compute sample variance $\hat{\sigma}^2(x)$ and compare to known threshold(s).

GPU Implementation Details

Precision Considerations:

- q can be large (e.g., $\log_2 q$ up to 50).
- Scaling data down for 16-bit floats is sufficient because we only need relative variance info.

PyTorch Compilation:

- Just-In-Time or graph-based optimization yields $2\text{-}3\times$ speedup vs. eager mode.

Throughput Example:

- On one NVIDIA V100, ~ 5 billion secret candidates ($n = 512$) tested in ≈ 1 hour.
- Bruteforce is repeated until the correct guess emerges (with high probability).

Need for Partial Lattice Reduction

Recall Attack Flow:

- 1 *Reduce* an embedded matrix Λ to obtain new $(\mathbf{A}', \mathbf{b}')$ with column variance partially compressed.
- 2 Identify high-variance columns (cruel) vs. low-variance columns (cool).
- 3 Brute force on cruel bits, then linear-time fix of cool bits.

Why partial?

- Overly strong reduction $\implies$ massive noise amplification, complicating distribution tests.
- Too weak reduction $\implies$ not enough columns are reduced (large n_u).

Generating Enough Reduced LWE Data

Sub-Sampling:

- If you start with m_0 LWE samples, choose subsets of size m_1 repeatedly.
- Each subset forms a new $(m_1 + n) \times (m_1 + n)$ q -ary lattice for reduction.
- This **inflates** the total number of partially reduced data sets, crucial when m is small.

SALSA Picante [7]:

- Eavesdropped $4n$ samples, sub-sampled n at a time for repeated BKZ runs.
- *This paper uses $m = \text{round}(0.875 n)$ from $4n$ total.*

Conclusion:

- Each sub-lattice reduced $\implies$ yields a distinct reduced matrix $\mathbf{A}'$.
- Attack = gather many $\mathbf{A}'$'s for large-scale statistical checks.

Choice of Reduction Algorithms

Three Tools:

- 1 BKZ2.0 [4]: strong reduction but slow for big block sizes/dimensions.
- 2 flatter [5]: fast for $n \geq 512$ and large q , but might get stuck.
- 3 polish [6]: final orthogonalization step after flatter or BKZ.

Measuring Performance:

$$\rho = \frac{\sigma(\mathbf{A}')}{\sigma(\mathbf{A}_{\text{initial}})} \quad \text{with} \quad \sigma(\mathbf{A}_{\text{initial}}) = \frac{q}{\sqrt{12}}.$$

- $\rho \ll 1 \implies$ large norm reduction.
- Attack wants ρ small but not so small that noise becomes unmanageable.

Interleaving & Early Termination

Interleaving BKZ & flatter:

- Run flatter for a few rounds.
- If average $\Delta\rho < 0.001$ over 3 iters, switch to BKZ2.0.
- After BKZ, run polish, then switch back to flatter, etc.

Early Termination:

- Once ρ stalls or improvements become tiny, export the reduced matrix $\mathbf{A}'$.
- Sub-sample a new set of LWE samples, repeat fresh.
- Maximizes the number of distinct partially reduced data sets from one heavy reduction session.

Result: Cruel Bits $\approx \rho^2 n$

- Empirically, $n_u \approx \rho^2 n$.
- Fewer cruel bits $\implies$ smaller dimension for brute force.
- If $\rho^2 n$ is still large, brute force is more expensive.

Trade-off Recap:

- ω (penalty param) and choice of algorithms $\implies$ final ρ .
- Must keep error amplification from $|\mathbf{Re}|$ under control, or distributions become too wide to distinguish.

Cruel Bit Verification: Sample Complexity

Distinguishing Uniform vs. Mod-Gaussian:

$$x = \mathbf{a} \cdot (s^* - s) - e \pmod{q}.$$

- If s^* has correct cruel bits, x has smaller variance $\sigma_{G_q}^2$.
- Otherwise, $x \approx$ uniform with variance $\sigma_{U_q}^2 \approx \frac{q^2}{12}$.

Variance Test: For M samples,

$$\hat{\sigma}^2 = \frac{1}{M} \sum x_i^2.$$

CLT-based Bound:

$$M(\alpha, \beta) \approx \left[\frac{G^{-1}(\alpha) \sigma_{U_q^2} + G^{-1}(\beta) \sigma_{G_q^2}}{\sigma_{G_q}^2 - \sigma_{U_q}^2} \right]^2,$$

controlling false negative α and false positive β errors.

Concrete M Values (Table 5)

n	$\log_2 q$	n_u	h	Worst Case M	Avg Case M
256	12	143	12	5.67×10^4	1.12×10^4
512	28	228	20	3.29×10^3	1.66×10^3
512	41	75	60	1.98×10^4	1.02×10^4
768	35	373	20	3.13×10^4	9.74×10^3
768	35	373	64	6.21×10^6	1.47×10^5
1024	50	495	15	2.37×10^3	1.35×10^3

Note: - Worst case: all secret 1-bits in reduced region. - Average case: 1-bits split proportionally. - M can be as low as a few thousand samples for some parameters or up to 10^6 for tough scenarios.

5.2 Testing Cool Bit Predictions

Algorithm 1 (Greedy) Recap:

- Flip each cool bit k from 0 to 1, measure $\hat{\sigma}^2(x_1)$ vs. $\hat{\sigma}^2(x_0)$.
- Keep the value that yields smaller residual variance.

Statistically:

$$\delta(\sigma^2) = \hat{\sigma}^2(x_1) - \hat{\sigma}^2(x_0).$$

- If bit k truly = 1, setting it to 1 *reduces* the variance in the presence of the correct preceding bits.
- Otherwise, it *increases* or doesn't lower variance enough, so keep it at 0.

Overall Results & Timings

Key Steps:

- 1 Lattice reduction (parallel): hours-scale for large n , but done once to generate multiple sub-samples.
- 2 Brute force on n_u cruel bits: total time depends on how quickly the correct candidate is found in ascending Hamming weight.
- 3 Cool bits: linear time via the greedy approach.

Table 6 (example):

- For $n = 512$, $\log_2 q = 41$, $h = 60$:
 - ≈ 6 minutes on 1 GPU for secret recovery (excluding lattice reduction).
 - Lattice reduction itself might take 12 hours/matrix but is fully parallelizable.
- Larger $n = 768$ or 1024 also feasible with multi-GPU setups.

Keynotes for Implementation and Performance

- **Parallel Sub-Sampling:**

- Increases number of reduced matrices from limited m_0 LWE samples.
- More reduced data = easier distribution testing.

- **Partial Lattice Reduction:**

- Balanced by penalty ω and the interleaving approach (BKZ + flatter).
- Achieves a sweet spot with $\rho \ll 1$ but not too large error amplification.

- **Cruel vs. Cool Recovery:**

- Brute force the small subset $n_u \approx \rho^2 n$, detect correctness by variance.
- Greedy fix the remaining n_r bits.

- **Efficient Implementation:**

- GPU parallelization + simple variance test.
- Feasible for $n \leq 1024$ even with high Hamming weight.

Concrete Performance Results

Overview (Refer to Table ??):

- After partial lattice reduction (Section 4), the main cost is enumerating over the “cruel” bits in ascending Hamming weight.
- We periodically apply the greedy approach (Algorithm 1) to recover the cool bits, e.g., every 40M secret candidates.
- Timing includes:
 - ~ 10 seconds of JIT compilation,
 - ~ 10 seconds of data loading,
 - GPU-based parallel brute force for the cruel portion.

Notes on Hamming Weights:

- Higher $h \implies$ typically more enumeration, or bigger chance h_u is large.
- Attackers can rely on multi-GPU setups (e.g., up to 20 GPUs for the toughest cases).

Table ??: Examples

- The table shows dimension n , $\log_2 q$, secret Hamming weight h , the number of bits in the cruel part h_u , and the fraction of secrets that produce that h_u (for random secrets).
- **Time (Min/Max)** depends on when the correct partial secret guess is found in the ascending-Hamming order.
- Typical sample usage: 200k for brute force, 1M for the greedy step at higher h settings.

Highlights:

- For $n = 256$, $\log_2 q = 12$, $h = 12$, some guesses complete in tens of seconds, while higher h_u can take thousands of seconds.
- $n = 512$, $\log_2 q = 41$, $h = 60$ may finish in a few hundred seconds for certain h_u but potentially longer for others.

Key Takeaway:

- The final recovery time strongly correlates with h_u (the 1-bits inside the cruel region).
- If h_u is small, the enumeration search is drastically faster.

Comparisons to Hybrid Dual MiTM, etc.

Why compare?

- The hybrid dual meet-in-the-middle (MiTM) approach [?] is conceptually closest.
- Some references [?], [?] provide partial results but mostly for $n < 110$ or smaller $\log q$.

Table ??:

- Shows the LWE Estimator predictions (ROP = number of ring ops).
- Compares them to actual code from [?] which times out or fails at $n > 256$.
- Our “Cool and Cruel” approach does scale to $n = 768$, $n = 1024$ with feasible timings.

Conclusion:

- Estimator-based ROP metrics can underestimate the real cost or do not translate well to GPU times.
- Actual meet-in-the-middle code [?] does not succeed beyond $n = 256$.

RLWE Basics

Ring-LWE Setting:

- $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $n = 2^k$.
- RLWE sample: $(a(x), b(x) = a(x) \cdot s(x) + e(x))$ corresponds to a skew-circulant matrix A_{circ} .
- Some RLWE rings can be attacked more easily than generic LWE [?], but 2-power cyclotomic rings remain a NIST standard candidate.

Our Attack:

- After partial reduction, columns still separate into cruel vs. cool.
- **Crucial difference:** We can rotate (shift) the cruel region across the ring polynomial, effectively searching multiple windows of length n_u .

Figure: LWE vs. RLWE Cost Ratio

Estimated $T_{\text{guess}}^{\text{LWE}}/T_{\text{guess}}^{\text{RLWE}}$ ratio for 10% sparse binary secret

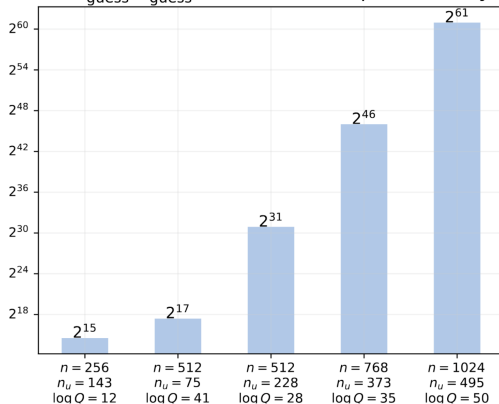


Fig.3 Example: Attack cost ratio $T_{\text{guess}}^{\text{LWE}}/T_{\text{guess}}^{\text{RLWE}}$ for 10% sparse binary secret, shown for ($n = 256, 512, 768, 1024$). Notice the exponential differences (e.g. $2^{31}, 2^{46}, 2^{61}$) as dimension grows.

RLWE Cruel Bit Shifting

Key Technique:

- We can shift the matrix A or the polynomial $a(x)$ so that different subsets of the secret s fall into the first n_u columns.
- Let $h_{n_u,i}(s) = \#$ of 1-bits in s over a length- n_u window starting at index i .
- We define $h_u^{(1)}(s) = \min_{0 \leq i < n} h_{n_u,i}(s)$, the minimal number of 1s in *any* length- n_u window.

Why Faster?

- If $h_u^{(1)}(s) \ll h_u(s)$, the enumerations become smaller by orders of magnitude.
- Potentially run n windows in parallel or pick the best (lowest-weight) window for brute force.

Table 8 Example: ($n = 256$, $\log_2 q = 12$, $h = 12$)

- Each row: a secret with $h_u(s)$ cruel bits (LWE viewpoint) vs. $h_u^{(1)}(s)$ after ring shifts.
- LWE times blow up quickly for large $h_u(s)$, sometimes not finishing (indicated by “-”).
- RLWE approach can exploit smaller $h_u^{(1)}$ windows, finishing in 10^3 or 10^4 seconds.

Findings:

- RLWE times $\approx 10^3 \times$ or more faster on average than LWE if ring shifting reveals a sufficiently small subset of bits.
- Estimated vs. actual times differ but confirm a wide gap in many tested secrets.

Table 9 Example: ($n = 512$, $\log_2 q = 41$, $h = 60$)

- Similar structure: 10 random secrets, each with a certain $h_u(s)$ for LWE and minimal $h_u^{(1)}(s)$ for RLWE.
- For large $h_u(s)$, the LWE brute force can be impractical, while RLWE shifting might find a window with only 3–5 bits.

Key Observations:

- Some secrets never complete under LWE enumeration ('-'), but RLWE recovers them due to partial shifting.
- The measured time can be 10^4 or 10^5 sec, significantly lower than the naive 10^7 or 10^9 sec if no shift is used.

RLWE vs. LWE: Overall Implications

- The ability to rotate the secret in 2-power cyclotomic RLWE leads to a **big advantage** for the attacker if the secret is sparse.
- In practice, partial lattice reduction cost is roughly the same, so the big gain is purely from enumerating fewer 1-bits in the cruel region.
- For real homomorphic encryption or ring-based cryptography, adopting small/sparse secrets can be *riskier* than previously assumed.

High-Level Conclusion:

- For $n = 256, 512, 1024$, the empirical data (Tables 8–9, Figure 3) confirm that “Cool and Cruel” attacks on RLWE can be up to 10^3 – 10^5 times faster compared to a standard LWE scenario with the same dimension/modulus.

Further Objectives

Summary of Main Findings

Cool & Cruel Attack Recap:

- Partial Lattice Reduction splits secret bits into *cruel* vs. *cool*.
- GPU-based brute force on cruel bits, variance-based or greedy approaches for cool bits.
- Demonstrated practicality up to $n = 1024$, especially for sparse secrets.
- RLWE (2-power cyclotomic) further lowers the dimension due to *cruel bit shifting*.

Open Questions:

- Large-scale brute force is still exponential.
- Lattice reduction vs. error amplification trade-off is not fully optimized.
- Sample-hungry approach for the cool bits remains a bottleneck when σ_e is large.

1. Improving the Combinatorial Scaling

Problem Statement:

- After partial reduction, we brute force n_u cruel bits.
- If n_u or the partial Hamming weight is large, enumeration can be very costly.

Potential Solutions:

① Meet-in-the-Middle (MiTM) / Hybrid Attacks:

- Split n_u bits into two halves, store partial sums, combine them to find matches.
- Reduces complexity from 2^{n_u} to $\sim 2^{n_u/2}$ with memory trade-offs.

② Heuristic Pruning or ML-Based Ranking:

- E.g., a small neural net or logistic model to discard unlikely bit patterns quickly.
- Inspired by SALSA-type or other learning-based approaches.

Approaches / Techniques:

- Hashed partial sums, advanced data structures, GPU kernels for MiTM.
- Minimal overhead “Early Abort” style checks or distribution-based

2. Balancing Reduction and Secret Recovery

Problem Statement:

- Stronger lattice reduction $\rightarrow$ fewer cruel bits, but higher error amplification σ_e .
- We do not currently co-optimize the partial reduction depth with enumeration cost.

Potential Solutions:

① Adaptive / Incremental Approach:

- Perform partial reduction in stages, attempt partial enumeration, then refine as needed.

② Search-Based Parameter Tuning:

- Model the total cost $T(\omega) = T_{\text{reduction}}(\omega) + T_{\text{enum}}(\omega)$, find ω or blocksize β that minimizes $T(\omega)$ in practice.

Approaches / Techniques:

- Heuristic optimization (simulated annealing, etc.), or real-time feedback on enumeration performance.
- Possibly do partial re-reduction of sub-lattices if the dimension is still too large.

3. Improving Cool Bit Recovery (Sample Efficiency)

Problem Statement:

- Current approach (Algorithm 1) flips each bit $\rightarrow$ measure variance $\sigma(\mathbf{A} \mathbf{s}^* - \mathbf{b})$.
- If σ_e is large, we may need many samples to distinguish flipping one bit at a time.

Potential Solutions:

① Machine Learning Distinguishers:

- Train a small classifier to evaluate correct vs. incorrect bit patterns without per-bit variance checks.

② Batch or Group Testing:

- Flip multiple bits simultaneously, analyze changes.
- Potentially reveal multiple bits in fewer samples.

Approaches / Techniques:

- Shallow neural networks or logistic regression for bit classification in reduced $\mathbf{A}'$.
- Carefully designed “group tests” (like combinatorial group testing) with distribution difference metrics

Additional Directions

Beyond the Above:

- **Ternary / Gaussian Secrets:** Test if the partial-lattice method extends to other secret distributions used in practice.
- **Combining with SALSA or Transformers:**
 - A synergy of partial reduction plus advanced ML might further reduce sample usage.
- **Real-World Impact / Mitigations:**
 - Evaluate big-name homomorphic encryption libraries to see if they inadvertently allow small h secrets or ring structures that are highly vulnerable.

Conclusion:

- **Cool & Cruel** is already practical, but many improvements remain to push it further or to defend against it.
- Future research will likely refine each stage (reduction, brute force, final bit checks) for greater efficiency.

Motivation for Multi-Zone Approach

Motivation: After partial lattice reduction, the standard deviation $\sigma(\mathbf{A}_{*,j})$ of columns often does *not* drop at a single index but rather transitions over a slope. Thus, we propose dividing columns into:

- **Cruel region** (green): High variance $\approx q/\sqrt{12}$.
- **Chill region** (yellow): Transitional/slope columns with moderate variance.
- **Cool region** (blue): Low variance, easily distinguished bits.

Goal: Use a *specialized* algorithm in the Chill zone to reduce brute force cost on partially reduced columns, while still using standard brute force for Cruel bits and a simple classification for fully Cool bits.

Key Observations:

- The boundary between these regions is determined by variance thresholds (or slope detection).
- Each zone is handled with a *tailored* approach (brute force, partial classification, or easy flips).

Annotated Figures: Cruel, Chill, and Cool Regions

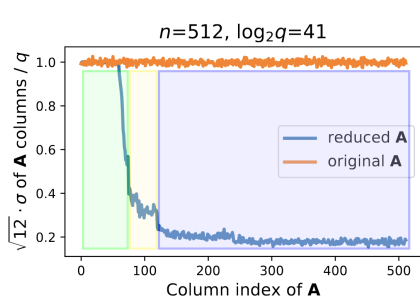
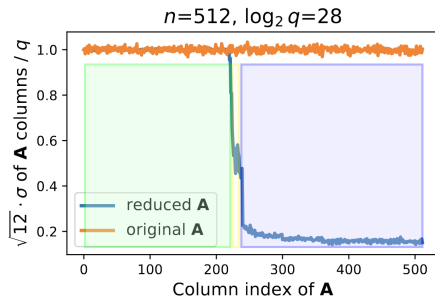
(a) $n = 512, \log_2 q = 41$.(b) $n = 512, \log_2 q = 28$.

Figure: Division of columns into **Cruel** (green), **Chill** (yellow), and **Cool** (blue) zones, reflecting standard deviation changes post-reduction.

Proposed Method: Threshold-Based Zoning

Threshold Model:

Let σ_j be the standard deviation of column j in the reduced matrix $\mathbf{A}'$. Define two thresholds T_c and T_h such that

$$T_c > T_h > 0, \quad (\text{e.g., } T_c \approx 0.8 \sigma_u, \quad T_h \approx 0.3 \sigma_u).$$

We classify each column j by:

$$\text{Region}(j) = \begin{cases} \text{Cruel} & \text{if } \sigma_j \geq T_c, \\ \text{Chill} & \text{if } T_h \leq \sigma_j < T_c, \\ \text{Cool} & \text{if } \sigma_j < T_h. \end{cases}$$

Why This Works:

- $\sigma_j \approx q/\sqrt{12} \rightarrow$ treat as Cruel, requiring heavy enumeration.
- $\sigma_j \ll q/\sqrt{12} \rightarrow$ treat as Cool, a small variance for easy per-bit flips.
- σ_j in-between $\rightarrow$ use a *more refined* partial approach.

Pseudocode: Multi-Zone Column Recovery

Objective: Merge brute force for Cruel bits, partial advanced method for Chill bits, and basic flips for Cool bits.

```

procedure MULTIZONERECOVERY(A, b,  $T_c$ ,  $T_h$ )
    zones  $\leftarrow \{\}$  ▷ Cruel, Chill, Cool region indices
    for  $j = 0 \rightarrow n - 1$  do
         $\sigma_j \leftarrow \text{StdDev}(\mathbf{A}_{*,j})$  ▷ post-reduction
        if  $\sigma_j \geq T_c$  then
            zones[Cruel]  $\leftarrow$  zones[Cruel]  $\cup \{j\}$ 
        else if  $\sigma_j \geq T_h$  then
            zones[Chill]  $\leftarrow$  zones[Chill]  $\cup \{j\}$ 
        else
            zones[Cool]  $\leftarrow$  zones[Cool]  $\cup \{j\}$ 
     $\mathbf{s}_{\text{cruel}} \leftarrow \text{BruteForce}(\mathbf{A}_{*,\text{zones[Cruel]}}, \mathbf{b})$  ▷ Exhaustive or hybrid search on
    cruel bits.
     $\mathbf{s}_{\text{cool}} \leftarrow \text{GreedyFlips}(\mathbf{A}_{*,\text{zones[Cool]}}, \mathbf{b})$  ▷ Simple bit-flip approach.
     $\mathbf{s}_{\text{chill}} \leftarrow \text{RefinedChill}(\mathbf{A}_{*,\text{zones[Chill]}}, \mathbf{b}, \mathbf{s}_{\text{cruel}}, \mathbf{s}_{\text{cool}})$  ▷ E.g., partial
    enumeration or advanced classifier.
    return  $\mathbf{s}_{\text{cruel}} \cup \mathbf{s}_{\text{chill}} \cup \mathbf{s}_{\text{cool}}$ 
  
```

Benefits and Challenges

Pros:

- Avoid over-enumerating columns that are “almost cool” but not entirely.
- Possibly reduce sample usage vs. forcing the entire slope region into a one-bit flip approach with large noise.
- Straightforward to implement if we define thresholds T_c, T_h carefully.

Cons / Open Questions:

- Choosing thresholds is somewhat heuristic. Slope detection or derivative-based methods might be more precise.
- REFINEDCHILL approach must be robust: a single misclassification can invalidate the overall guess.
- If the slope is wide, partial enumeration might still be too big, requiring further sub-lattice re-reductions or advanced pruning.

Future Steps:

- Experiment with different ω (penalty) to see how the slope changes in dimension n .

References

-  O. Regev. *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography*. J. ACM 51(6), 2005.
-  Niklas N., ... *The Cool and the Cruel: Separating Hard Parts of LWE Secrets.*, <https://eprint.iacr.org/2024/443.pdf> (ArXiv, 2024).
-  Albrecht et al. *Estimate all the LWE, NTRU schemes.* <https://eprint.iacr.org/2018/331> (2018).
-  Chen, Y., Nguyen, P. Q., *BKZ 2.0: Better Lattice Security Estimates*. Lecture Notes in Computer Science, 2011.
-  Ryan, K., Heninger, N. *Fast practical lattice reduction through iterated compression*. Annual International Cryptology Conference. Springer, 2023.
-  Charton, F., Lauter, K., Li, C., Tygert, M. *An efficient algorithm for integer lattice reduction*. *SIAM Journal on Matrix Analysis and Applications* (2024).
-  Gama, N., Ducas, L. et al., *On SALSA Picante ML Attack Method*. <https://eprint.iacr.org/2023/340> (2023).